

Trabajo Práctico N 10 - Taller de Arquitectura

Implementación de funcionalidades al procesador TDA 1819

Introduccion

El presente trabajo tiene como objetivo implementar las funcionalidades del procesador TDA1819 para el manejo de la pila y la modificaciones de varios componentes en el set de instrucciones existente. Se implementaron las instrucciones pushh y poph, destinadas a realizar el apilado y desapilado respectivamente, así como una funcionalidad adicional que permite acceder a la pila sin necesidad de realizar dichas operaciones.

Además, se desarrolló un código en lenguaje Assembly, para el uso de dichas implementaciones, utilizando datos como DNI y legajo para realizar operaciones en la pila y volcar los resultados en memoria principal.

Para la validación, se simuló el programa en el procesador, tanto con segmentación habilitada, como no, pudiendo determinar el tiempo total de ejecución, y para cada instrucción, el comportamiento del registro SP y de la memoria de la pila. En este informe, se citará código modificado y se mostrarán fragmentos representativos, junto a una explicación.

Ejercicio 1

Implementacion pushh y poph

En este punto, se busca implementar las instrucciones de manejo de pila para half words. Las instrucciones permiten apilar y desapilar datos de 16 bits en la sección de memoria asignada a la pila. Se decidió que la pila crezca de manera descendente con respecto a las direcciones de memoria, por lo que, ocupando la memoria de pila el espacio comprendido entre la dirección 0x7000 y 0x8000.

Un detalle a tener en cuenta es que para esta implementación no se tuvieron en cuenta comprobaciones lógicas en lo referido al manejo de pila. Por ejemplo, hacer poph sin que exista pila, que el SP pase a menos de 0x7000, etc.

Ensamblado

Para implementar las instrucciones pushh y poph fue necesario adaptar el ensamblador para que reconozca estas nuevas instrucciones, asigne su código de operación (opcode) y reserve el espacio correspondiente en la memoria de instrucciones. En concreto se realizaron cambios en los paquetes de tipos y constantes y en la lógica de análisis de texto del ensamblador que determina si una cadena de entrada corresponde a una instrucción del conjunto de manejo de pila.

Los pasos seguidos fueron:

1. Añadir nuevos tipos y arreglos para almacenar nombres, códigos y tamaños de las instrucciones de manejo de pila.
2. Declarar constantes que contienen los strings de las instrucciones ("pushh", "poph"), sus códigos (PUSHH, POPH) y el tamaño en bytes de cada instrucción.
3. Extender el procedimiento de verificación (checkCode) para invocar checkInstMp, que detecta las instrucciones de pila, asigna el opcode y actualiza addr_linea con el tamaño correspondiente.

A continuación se muestran fragmentos de código con las modificaciones realizadas. Cada fragmento incluye una breve explicación de su función y el motivo del cambio.

Fragmento de código representativo de lo que fue modificado en src/Packages/tipos_ensamblador.vhd. Se agregaron los nuevos tipos de instrucciones de manejo de pila.

```
--- Añado tipos para la implementación de las instrucciones pushh y poph ---  
  
TYPE instmp_name_array IS ARRAY (POSITIVE RANGE <>) OF STRING(1 to 5);  
TYPE instmp_code_array IS ARRAY (POSITIVE RANGE <>) OF STD_LOGIC_VECTOR(7 downto 0);  
TYPE instmp_size_array IS ARRAY (POSITIVE RANGE <>) OF INTEGER;
```

Fragmento de código representativo de lo que fue modificado en src/Packages/const_ensamblador.vhd. Se especifican los nombres, códigos de operación y tamaño de las nuevas instrucciones de manejo de pila.

```
--- Añado constante para la implementación de las instrucciones pushh y poph ---  
  
CONSTANT CANT_INSTMP:      INTEGER := 2;  
  
-----  
  
--- Añado constantes para la implementación de las instrucciones pushh y poph ---  
  
CONSTANT INSTMP_NAMES:     instmp_name_array(1 to CANT_INSTMP) := ("pushh", "poph ");  
CONSTANT INSTMP_CODES:     instmp_code_array(1 to CANT_INSTMP) := (PUSHH, POPH);  
CONSTANT INSTMP_SIZES:     instmp_size_array(1 to CANT_INSTMP) := (3, 3);
```

Fragmento de código representativo de lo que fue modificado en src/Usuario/PC/1. Ensamblador/ensamblador.vhd. En el procedimiento checkCode se incorpora una llamada al nuevo procedimiento checkInstMp (ver completo en código), que verifica si la instrucción actual pertenece al conjunto de instrucciones de manejo de pila.

```
--- Añado checkInst para Manejo de Pila para la implementación de las instrucciones pushh y poph ---  
  
PROCEDURE checkInstMp(  
  CONSTANT cadena      : IN STRING;  
  CONSTANT INSM DP_NAME : IN STRING;  
  CONSTANT INSM DP_CODE : IN STD_LOGIC_VECTOR(7 downto 0);  
  CONSTANT INSM DP_SIZE : IN INTEGER;  
  i                : INOUT INTEGER;  
  check            : INOUT BOOLEAN;  
  CONSTANT nombre     : IN STRING;  
  CONSTANT num_linea  : IN INTEGER;  
  CONSTANT addr_linea : IN INTEGER  
) IS  
  VARIABLE match      : BOOLEAN := true;  
  VARIABLE indice     : INTEGER := i;  
  VARIABLE numReg1    : INTEGER;  
BEGIN
```

PC

Ahora que el ensamblador reconoce correctamente las nuevas instrucciones pushh y poph, se debe modificar el resto de la computadora, para que la misma implemente las funcionalidades esperadas.

Cuando el procesador recibe una instrucción (pushh o poph) deberán modificarse componentes como: memoria, registro SP, registro destino, etc. Por lo que, se decidió implementar el cambio del registro SP desde la etapa ID, basado en el caso de la instrucción de salto incondicional JMP, la cual modifica en esta misma etapa el registro IP . Así como hay una señal dedicada para modificar el registro IP (EnableRegIDIP), se agregó la señal EnableRegIDSP para permitir a la etapa de ID modificar el SP.

Una vez recibida la instrucción, se comprueba que no haya atasco RAW, y en ese caso, se modifica el SP y se ordena a la etapa WB modificar el registro destino en caso de que corresponda.

Para la memoria, se agregó al espacio de direcciones de datos las direcciones correspondientes a la memoria de pila (comprendida entre 0x7000 y 0x8000).

Aclaración: opte por no mostrar fragmentos de código, ya que los mismos eran extensos comparados a los anteriores. Todas las modificaciones que se han hecho se encuentran en el código fuente.

Ejercicio 2

Para la lectura y escritura en la pila, se decidió adaptar las instrucciones shh y lh para que puedan aceptar, al registro fuente SP y al valor Inm, a un número literal y no únicamente etiquetas. Se modificó la lógica interna del ensamblador: cuando se verifica si el string en el lugar del operando Inm coincide con algunas de las variables, se agregó que en caso de que no coincida con ninguna, en vez de dar error, verifique primero si el string está compuesto únicamente por números, ya que ese sería el caso donde se trata de un valor literal. Además, para poder permitir que SP sea registro fuente, se adaptó la verificación lógica del string, separando los casos en los que el registro fuente es un registro de uso general a cuando es el SP.

Adjunto algunos fragmentos de código:

```
--- Añado lógica para detectar numeros literales como Inm ---
-- debe haber al menos un dígito
if (not isNumber(cadena(indice))) then
    report "Error en la línea " & integer'image(num_linea) &
        " del programa '" & trim(nombre) &
        "': el segundo operando no coincide con una variable ni es un inmediato válido"
    severity FAILURE;
end if;

-- parsear número decimal de varias cifras
addrInm := 0;
while (indice <= cadena'length and isNumber(cadena(indice))) loop
    for j in DIGITS_DEC'range loop
        if (cadena(indice) = DIGITS_DEC(j)) then
            addrInm := addrInm * 10 + (j-1);
            exit;
        end if;
    end loop;
    indice := indice + 1;
end loop;

--- Añado lógica para detectar SP como registro fuente ---
if (cadena(indice) = 's') then
    indice := indice + 1;
    if (cadena(indice) /= 'p') then
        report "Error en la línea " & integer'image(num_linea) & " del programa '" & trim(nombre) & "': el segundo operando se encuentra incorrectamente declarado"
        severity FAILURE;
    end if;
    indice := indice + 1;
    if (cadena(indice) /= 'l') then
        if (cadena(indice-1) /= '1') then
            report "Error en la línea " & integer'image(num_linea) & " del programa '" & trim(nombre) & "': el segundo operando se encuentra incorrectamente declarado"
            severity FAILURE;
        end if;
        case cadena(indice) is
            when '0' to '5' =>
                for j in DIGITS_DEC'range loop
                    if (cadena(indice) = DIGITS_DEC(j)) then
                        addrReg := 10 + j-1;
                        exit;
                    end if;
                end loop;
                indice := indice + 1;
            when others =>
                report "Error en la línea " & integer'image(num_linea) & " del programa '" & trim(nombre) & "': el segundo operando se encuentra incorrectamente declarado"
                severity FAILURE;
        end case;
    end if;
    if (cadena(indice) /= 'l') then
        report "Error en la línea " & integer'image(num_linea) & " del programa '" & trim(nombre) & "': el segundo operando se encuentra incorrectamente declarado"
        severity FAILURE;
    end if;
    addrReg := ID_SP;
    -- SP apunta en la prox direccion vacia de la pila a escribir, por eso sumo 1
```

Nuevamente, en el código fuente se puede ver la lógica completa del desarrollo.

Ejercicio 3

El código Assembly solicitado, hace uso de las instrucciones de manejo de pila implementadas junto con el mecanismo de acceso a la misma. A continuación, adjunto el código y el cálculo de los valores A y B. Valor A: 3207, derivado del número de legajo 03207/3. Valor B: 8496, derivado del dni 44.798.496

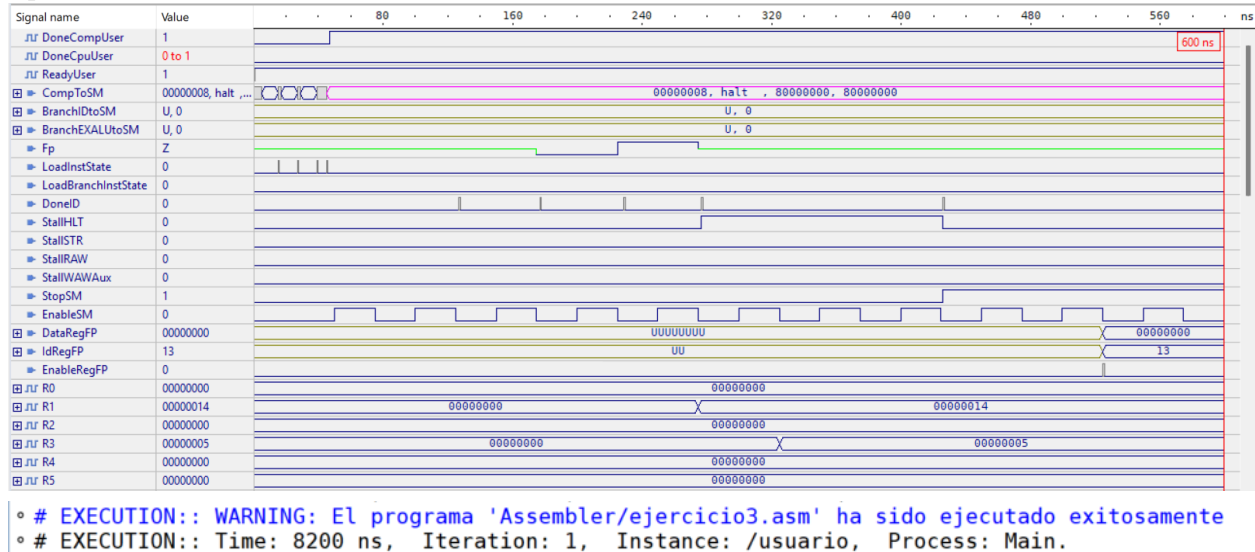
```
.data
A:    .hword    3207
B:    .hword    8496
SUMA: .hword    0
MAYOR: .hword    0
MENOR: .hword    0
.code
    lh r1, A(r0)
    lh r2, B(r0)
    slt r5, r1, r2
    andi r5, r5, 1
    bnez r5, Amenor
Bmenor:    daddi r10, r2, 0
            daddi r11, r1, 0
            jmp seguir
Amenor:    daddi r10, r1, 0
            daddi r11, r2, 0
seguir:    dadd r3, r1, r2
            pushh r3
            andi r4, r3, 1
            beqz r4, ESPAR
IMPAR:    pushh r10
            pushh r11
            jmp incisoh
ESPAR:    pushh r11
            pushh r10
incisoh:    lh r6, 4(sp)
            sh r6, SUMA(r0)
            lh r7, 2(sp)
            lh r8, 0(sp)
            slt r9, r7, r8
            andi r9, r9, 1
            bnez r9, sietemnr
            sh r7, MAYOR(r0)
            sh r8, MENOR(r0)
            jmp incisoi
sietemnr:    sh r8, MAYOR(r0)
            sh r7, MENOR(r0)
incisoi:    beqz r4, ESPARP
            daddi r12, r0, 1
            sh r12, 4(sp)
            poph r11
            poph r10
            jmp fin
ESPARP:    sh r0, 4(sp)
            poph r10
            poph r11
fin:    poph r3
            halt
```

Ejercicio 4

A continuación, muestro fragmentos de la simulación, indicando el tiempo de ejecución para cada caso.

Aclaración: Al momento de la simulación, no entiendo por qué razón, ya que antes me funcionaba bien, tuve problemas para visualizar los valores en el WaveForm. Los tiempos de simulación los toma correctamente, no así los valores visualizados.

Pipeline desactivado



Pipeline activado

